

Stochastic Neural Models

John Snyder

STAT 9340: Spring 2023

Section 1

Deep Gaussian Processes

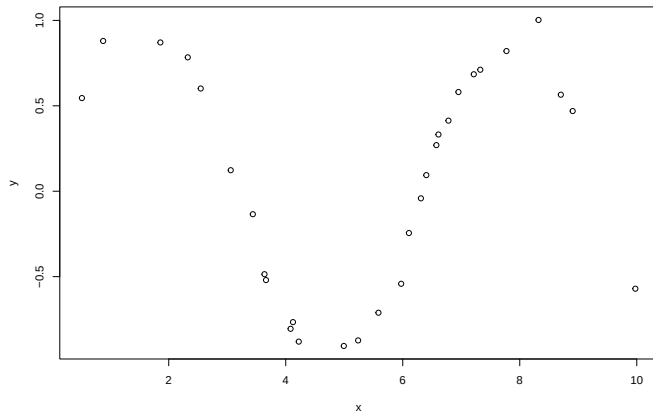
Gaussian Processes (GPs)

Recall from Data III, say we are interested in flexibly predicting a variable of interest, say Y , given some inputs, X . We may also have measurement noise in our observation of Y , which we call Z . The Gaussian Process (GP) approach to this problem makes use of the flexibility of assuming that Y is a random (Gaussian) process that is controlled by a mean function and covariance function. This functional assumption allows us to predict Y beyond where we have observations.

GPs were first used in spatial statistics starting in the 1960s to do optimal spatial interpolation (e.g., the famed “kriging” methodology) but has in the last decade or so been used in machine learning to do more general prediction.

Gaussian Process Regression: Example

Say we want to predict at x locations that were not observed.



Data Model for Prediction

We represent our response in terms of the latent process of interest plus an observation error:

$$z_i = Y_i + \epsilon_i, \quad i = 1, \dots, n$$

Assumptions:

- Y is a latent process specified by a GP (see below)
- $\{\epsilon_i\}$ - independent and identically distributed mean-zero observation errors that are independent of Y and have variance σ_ϵ^2
- the errors are assumed to have a normal (Gaussian) distribution here; $\epsilon_i \sim \text{iid } N(0, \sigma_\epsilon^2)$ (we might also write this as $\epsilon_i \sim \text{iid Gau}(0, \sigma_\epsilon^2)$); more generally, these errors could be correlated as well

The Purpose of Linear Prediction

We seek to predict the “true” (latent) process given any x , say (x_0) for which we might not have an observation of Y , say $Y(x_0)$, given all of the observations $\{z_i : i = 1, \dots, n\}$. Consider a linear predictor of the form:

$$\hat{Y}(x_0) = k_0 + \sum_{i=1}^n w_i z_i,$$

where w_i are “weights” and k_0 is some constant. We need to choose these weights. How? Instead of ad hoc approaches, we want to find the “optimal” weights. This requires that we consider the random process that controls Y more carefully.

Latent Process Model

We assume that the latent process follows the model

$$Y(x) = f(x)$$

where

$$f(x) \sim GP(m(x), c(x, x'))$$

is a random process, and

- $m(x)$ is the mean function, $E[f(x)]$, and
- $c(x, x')$ is the covariance function,
$$c(x, x') = E[(f(x) - m(x))(f(x') - m(x'))^T].$$

Gaussian Processes

What is the difference between a Gaussian “distribution” and a Gaussian “process”?

- Gaussian Distribution: a distribution over vectors
 - ▶ Fully specified by a mean vector and covariance matrix: e.g.,
 $\mathbf{Y} \sim \text{Gau}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ (i.e., a multivariate normal distribution)
- Gaussian Process: a distribution over functions
 - ▶ Fully specified by a mean function (m) and covariance function (c): e.g.,
 $Y \sim GP(m, c)$; thus, we will need to specify a mean function and covariance function to define a GP

Gaussian Processes (cont)

Why GPs?

- A GP is an infinite-dimensional object
- This will allow us to make a prediction anywhere in our x domain (if we know the mean and covariance functions); as we will see, we need the covariance between ANY two x locations
- But, in practice we will only ever need to consider finite-dimensional distributions; why?
 - ▶ We only have a finite set of data and are eventually only interested in a finite set of predictions
 - ▶ Any finite collection of GP random variables has a joint Gaussian distribution
 - ▶ This allows us to use the traditional machinery of multivariate normal distributions and linear mixed models to do calculations

GP Regression: Optimal Linear Prediction

Let's get back to the problem at hand: Recall we are interested in predicting $Y(x_0)$ (i.e., at some arbitrary x) given the data $\mathbf{z} \equiv (z_1, \dots, z_n)^T$. Assume for simplicity that the mean function $\mu(x)$ is known for all x (e.g., a constant) [we can relax this and make it a function of covariates].

We seek linear predictors that minimize the mean squared prediction error of the form

$$\hat{Y}(x_0) = k(x_0) + \sum_{i=1}^n w_i(x_0) z_i = k(x_0) + \mathbf{w}_0^T \mathbf{z}.$$

The optimal linear predictor in this case is also the conditional expectation $E[Y(x_0) | \mathbf{z}]$. Thus, we seek the conditional distribution $[Y(x_0) | \mathbf{z}]$.

GP Regression: Optimal Linear Prediction

It helps to rewrite the data model in vector form:

$$\mathbf{z} = \mathbf{Y} + \varepsilon, \quad \varepsilon \sim \text{Gau}(\mathbf{0}, \mathbf{C}_\varepsilon)$$

where $\mathbf{Y}$ and ε are ordered in the same way as the data vector $\mathbf{z}$. Now, $\text{cov}(\mathbf{Y}, \mathbf{Y}) \equiv \mathbf{C}_y$, $\text{cov}(\varepsilon, \varepsilon) \equiv \mathbf{C}_\varepsilon$, $\text{cov}(\mathbf{z}, \mathbf{z}) = \mathbf{C}_y + \mathbf{C}_\varepsilon$. In addition, $\mathbf{c}_0^T \equiv \text{cov}(Y(x_0), \mathbf{z})$, $c_{0,0} \equiv \text{var}(Y(x_0))$, μ_0 is mean at x_0 and $\boldsymbol{\mu}$ is the mean corresponding to the observations. Then, consider the joint Gaussian distribution:

$$\begin{bmatrix} Y(x_0) \\ \mathbf{z} \end{bmatrix} \sim \text{Gau} \left(\begin{bmatrix} \mu_0 \\ \boldsymbol{\mu} \end{bmatrix}, \begin{bmatrix} c_{0,0} & \mathbf{c}_0^T \\ \mathbf{c}_0 & \mathbf{C}_z \end{bmatrix} \right)$$

GP Regression: Optimal Linear Prediction

Using the joint/conditional Gaussian distribution formula from before we get

$$Y(x_0) | \mathbf{z} \sim \text{Gau} \left(\mu_0 + \mathbf{c}_0^T \mathbf{C}_z^{-1} (\mathbf{z} - \boldsymbol{\mu}), c_{0,0} - \mathbf{c}_0^T \mathbf{C}_z^{-1} \mathbf{c}_0 \right)$$

The prediction formula is given by the mean of this distribution:

$$\hat{Y}(x_0) = \mu_0 + \mathbf{c}_0^T \mathbf{C}_z^{-1} (\mathbf{z} - \boldsymbol{\mu})$$

and the prediction variance is then $\sigma_{Y_0}^2 = c_{0,0} - \mathbf{c}_0^T \mathbf{C}_z^{-1} \mathbf{c}_0$. It can be shown that this predictor is the optimal predictor in that it minimizes the mean squared prediction error:

$$MSPE = E \left[\left(Y(x_0) - \hat{Y}(x_0) \right)^2 \right]$$

GP Regression: Optimal Linear Prediction

Thus, in this case, the weights are given by $\mathbf{w}_0 = \mathbf{c}_0' \mathbf{C}_z^{-1}$ and $k(x_0) = \mu_0 - \mathbf{c}_0' \mathbf{C}_z^{-1} \boldsymbol{\mu}$. These imply an effective kernel, just as we saw with splines and local regressions.

In practice, one can easily adapt these formulas to get the optimal prediction for multiple locations at once (each location has its own vector of weights). It is also simple to implement this in the case where there are no measurement errors, i.e., $z_i = y_i$ (essentially, replacing $\mathbf{C}_z$ with $\mathbf{C}_y$). Note that if there are a lot of sample locations, it can be problematic to compute the matrix inverse, $\mathbf{C}_z^{-1}$. However, there are ways to approximate this (e.g., nearest-neighbor Gaussian processes).

Covariance Functions

To be able to predict at new x locations, we must specify a mean function (easy) and a covariance function between ANY two locations (not so easy). So far, we assumed that we knew the variances and covariances that make up $\mathbf{C}_y$, $\mathbf{C}_\epsilon$, $\mathbf{c}_0$, and $c_{0,0}$. In the real-world we would rarely (if ever) know these.

Covariance Functions

Seemingly simple solution: specify covariance functions in terms of parameters, θ , that can give a covariance between any pairs of coordinates, and then estimate the parameters. This is essentially what we did in linear mixed models.

- Covariance functions must be positive semi-definite (this ensures that prediction variances are non-negative!)
- Covariance functions should be realistic (that is, they should account for realistic dependencies) [this can be difficult, depending on the application, but may not matter in some cases]; typically, it is sufficient to assume that “nearby” observations are more dependent.
- Typically make assumptions to help: e.g., stationarity, isotropy, separability, and select from known classes (e.g., exponential, Gaussian, Matérn, power, etc.); e.g., the exponential covariance function is given by

$$c(\tau; \theta) = \sigma^2 \exp \left\{ -\frac{|\tau|}{\theta} \right\},$$

where σ^2 is the variance parameter and θ is the dependence parameter.

Estimation

Of course, we need to estimate the parameters in the covariance functions, and in the mean function if there are extra covariates.

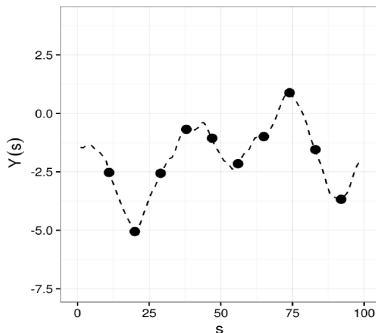
From a frequentist perspective, this can be done as we discussed in linear mixed models. That is, the parameters can be estimated through REML or MLE, and the random process can be predicted based on the BLUP.

In machine learning, one often takes a Bayesian perspective with GP implementations. In that case, the GP for $f(x)$ is considered the prior distribution, and one updates this with the observations, z .

As mentioned, in situations where there are a lot of data locations, REML, MLE, and Bayesian implementations can be problematic because of the presence of the high-dimensional covariance matrix, $\mathbf{C}_z$ or $\mathbf{C}_y$. Making these implementations efficient in “big data” settings has been the subject of intense research over the last 10-15 years. These issues are compounded for non-Gaussian likelihoods and latent GP processes. In machine learning, variational approaches have been the norm.

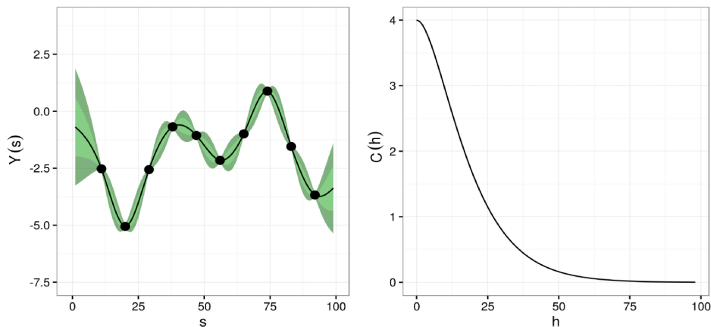
Example: 1-d

Consider a simulated process $Y(s)$; data are the dots and true process is the dashed line. This example assumes no measurement error.



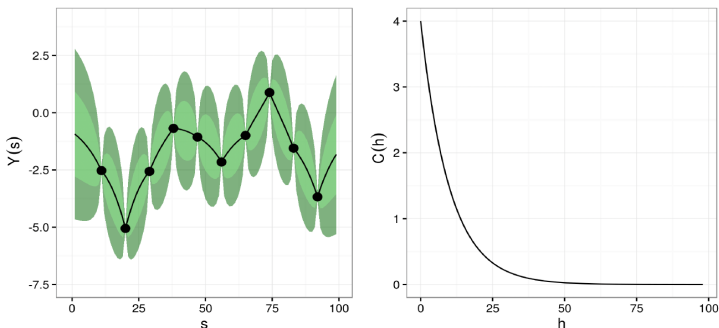
Example: 1-d

Predictions and prediction intervals with the covariance function shown on the right (a Matérn covariance function).



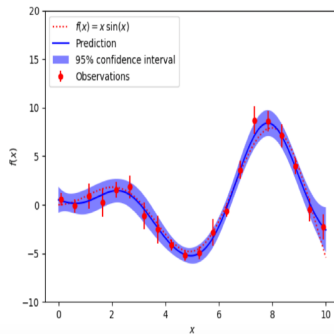
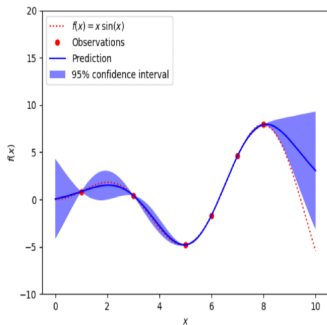
Example: 1-d

Predictions and prediction intervals with the covariance function shown on the right (an exponential covariance function).



Example: 1-d

Left: no measurement error; Right: includes measurement error.



Hierarchical GPs

Consider one way to write a multi-level (hierarchical) GP model utilizing conditional means.

$$\begin{aligned}\mathbf{Z} \mid \mathbf{Y}^{(k)} &\sim \text{Gau}(\mathbf{Y}^{(k)}, \boldsymbol{\Sigma}_{\epsilon}) \\ Y^{(k)}(x) \mid Y^{(k-1)}(x) &\sim GP(Y^{(k-1)}(x), c(x, x', \boldsymbol{\theta}^{(k-1)})) \\ Y^{(k-1)}(x) \mid Y^{(k-2)}(x) &\sim GP(Y^{(k-2)}(x), c(x, x', \boldsymbol{\theta}^{(k-2)})) \\ &\vdots \\ Y^{(2)}(x) \mid Y^{(1)}(x) &\sim GP(Y^{(1)}(x), c(x, x', \boldsymbol{\theta}^{(1)}))\end{aligned}$$

One could get the posterior by Bayes rule:

$$\left[Y^{(k)}(\cdot), \dots, Y^{(1)}(\cdot), \boldsymbol{\theta}^{(k)}, \dots, \boldsymbol{\theta}^{(1)} \mid \mathbf{Z} \right] \propto \left[\mathbf{Z} \mid \mathbf{Y}^{(k)}, \boldsymbol{\theta}^{(k)} \right] \dots \left[Y^{(2)}(\cdot) \mid Y^{(1)}(\cdot), \boldsymbol{\theta}^{(1)} \right]$$

Hierarchical GPs

In principle, this would be pretty easy to implement by MCMC.

Indeed, such hierarchical models have been used for complex environmental processes for years.

One thing to note here is that the GPs are still all operating on x , although in principle other covariates could come into the mean functions easily enough.

Although this is a “deep” model, it is not really what is meant by a deep GP. Let's take a look at another representation.

Deep Gaussian Processes

As described in Damianou and Lawrence (2013), a deep GP “consists of a cascade of hidden layers of latent variables where each node acts as output for the layer above and as input for the layer below - with the observed outputs being placed in the leaves of the hierarchy. Gaussian processes govern the mappings between the layers.”

On the surface, the hierarchical GP model we wrote above would satisfy this. The difference is how the output from one GP maps into the other. In the context of deep GPs, the outputs of one layer act like the x 's in the formulation above.

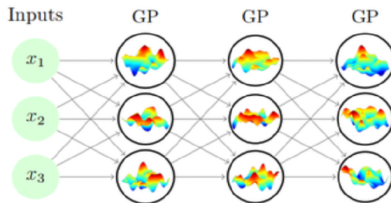
This adds significant complexity because the GP input is now impacting the current GP in a very nonlinear way.

Deep GPs (DGPs)

DGPs learn the function mapping of the input sentences to the output classes using composition of functions represented as:

$$f(\mathbf{x}) = f^L \circ \left(f^{L-1} \dots \circ \left(f^1(\mathbf{x}) \right) \right)$$

for L layers. Note, the ℓ -th layer comprises D^ℓ functions, $f^\ell = \{f_j^\ell\}_{j=1}^{D^\ell}$, which maps the representations obtained from the previous layer, $\ell - 1$ to obtain D^ℓ representations for layer ℓ . E.g.,



For every j th representation and every layer ℓ , independent GP priors are used. For example, the function f_j^ℓ has prior $f_j^\ell(\cdot) \sim GP(m_j^\ell(\cdot), k^\ell(\cdot, \cdot))$. The j th function in layer 1, f_j^1 , is evaluated on the input data point $\mathbf{x}_i$ to obtain $F_{i,j}^1 = f_j^1(\mathbf{x}_i)$. Extending this to build the hierarchy of layers, the j th function of layer ℓ , $f_j^\ell(\cdot)$ is evaluated on the data representation $\mathbf{x}_i$ obtained (transformed) from the previous layer, $\ell - 1$, $F_i^{\ell-1}$, so, $F_{i,j}^\ell = f_j^\ell(F_i^{\ell-1})$. Let $\mathbf{f}_j^\ell$ denote the j th representation of layer ℓ computed across all inputs. The final layer, L , will have K functions with respect to all the output classes and flattening of these functions using a softmax (classification) or identity (regression) function.

Hyperparameters for the covariance functions are learned via variational methods (e.g., see Salimbeni and Deisenroth, 2017, for details). This is where the difficult challenges have been with implementing these models. DGPs have been applied to many different learning problems - they have an advantage that uncertainty quantification is naturally part of the formulation.

DGPs in Python

The `gpflow`/`gpflux` python ecosystem can fit DGPs using Tensorflow as a backend.

- GPflow is a general purpose GP library
- GPflux facilitates the building of DGPs models with TensorFlow.
- Ideal because this can utilize GPUs for model fitting.

In R: The `deepgp` package

- Seems straightforward to use, but not very flexible

Steps of fitting a DGP model with gpflux

For *each* layer we want to fit

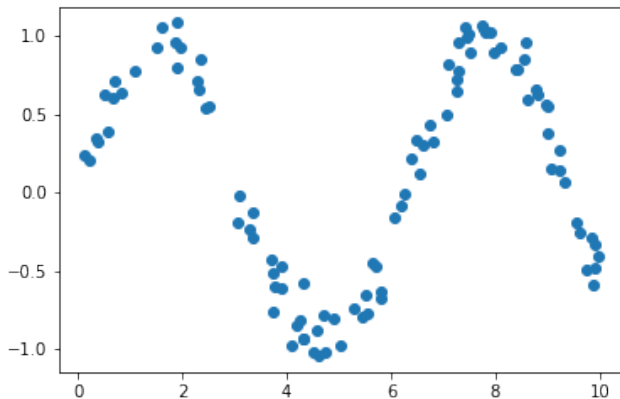
- ① Define the kernel function: The kernel function specifies the covariance between two input points.
 - Radial Basis Function (RBF), Matern, Exponential, etc.
- ② Define the inducing points: A set of points in the input space that are used to represent the full input space. They are often used in the context of sparse Gaussian processes, because the full covariance matrix is often too large to work with.

Then, we finally

- ③ Define the likelihood function: The likelihood function specifies the distribution over the observed data given the latent function values.
 - Output data, gaussian, poisson, etc

Define some data to get going

```
X = np.random.rand(100, 1) * 10  
Y = np.sin(X) + np.random.normal(0, 0.1, (X.shape[0],1))
```



Define the model

```
# Define the deep GP model
num_data, d_xim = X.shape
num_inducing = 20 # number of inducing points

inducing_variable = gpflow.inducing_variables.InducingPoints(
    np.linspace(X.min(), X.max(), num_inducing).reshape(-1, 1)
    #X.reshape(-1,1)
)

kernel = gpflow.kernels.Exponential()

gp_layer = gpflux.layers.GPLayer(
    kernel, inducing_variable,
    num_data=num_data, num_latent_gps=1
)

likelihood_layer = gpflux.layers.LikelihoodLayer(
    gpflow.likelihoods.Gaussian(0.1))
```

Fitting the model

This is similar to the final step of defining a Tensorflow model, but there are two key arguments

- `f_layers`: The layers $[f_1, f_2, \dots, f_n]$ describing the latent function $f(x) = f_n(\dots(f_s(f_1(x))))$.
- `likelihood`: The layer for the likelihood $p(y|f)$.

```
single_layer_dgp = gpflux.models.DeepGP([gp_layer],
                                         likelihood_layer)

model = single_layer_dgp.as_training_model()
model.compile(tf.optimizers.Adam(0.01))

model.fit({"inputs": X, "targets": Y},
         epochs=int(1000), verbose=0)
```

Plotting the predictions

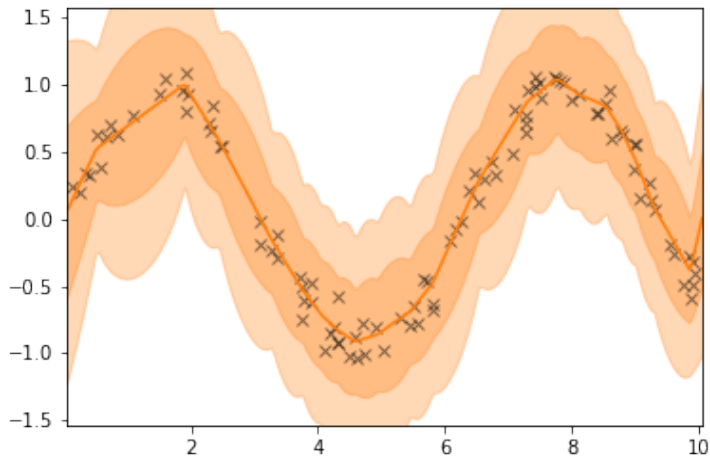
```
X_MARGIN, Y_MARGIN = 0.1, 0.5
fig, ax = plt.subplots()
num_data_seq = 200
X_seq = np.linspace(X.min() - X_MARGIN,
                    X.max() + X_MARGIN,
                    num_data_seq).reshape(-1, 1)
model = single_layer_dgp.as_prediction_model()
out = model(X_seq)

mu = out.y_mean.numpy().squeeze()
var = out.y_var.numpy().squeeze()
X_test = X_seq.squeeze()

# dual bands
for i in [1, 2]:
    lower = mu - i * np.sqrt(var)
    upper = mu + i * np.sqrt(var)
    ax.fill_between(X_test, lower, upper,
                    color="C1", alpha=0.3)

ax.set_ylim(Y.min() - Y_MARGIN, Y.max() + Y_MARGIN)
ax.set_xlim(X.min() - X_MARGIN, X.max() + X_MARGIN)
ax.plot(X, Y, "kx", alpha=0.5)
ax.plot(X_test, mu, "C1")
```


Plotting the predictions(cont)



Deep Gaussian Process on College data

```
College = pd.read_csv('.../College.csv')
XCollege = College[['Top10perc', 'PhD']].values

YCollege = 100*(College['Accept']/College['Apps']).values.reshape(-1)
X, X_test, Y, Y_test = train_test_split(XCollege, YCollege,
                                         test_size=0.1,
                                         random_state=1)

scaler = StandardScaler()
scaler.fit(X)

# Transform the training and test data
X_scaled = scaler.transform(X)
X_test_scaled = scaler.transform(X_test)

num_data, d_xim = X.shape
```

The difference here is the multidimensional input.

- This will require us to specify multiple columns for our InducingPoints

Layer 1

```
kernel = gpflow.kernels.SquaredExponential()
num_inducing = 100 # number of inducing points

inducing_variable = gpflow.inducing_variables.InducingPoints(
    np.concatenate(
        [
            np.linspace(X[:,0].min(), X[:,0].max(),
                        num_inducing).reshape(-1, 1),
            np.linspace(X[:,1].min(), X[:,1].max(),
                        num_inducing).reshape(-1, 1)
        ],axis=1)
    )
gp_layer = gpflux.layers.GPLayer(
    kernel,
    inducing_variable,
    num_data=num_data,
    num_latent_gps=1,
    mean_function=gpflow.mean_functions.Zero(),
)
```

Layer 2 and output

```
kernel = gpflow.kernels.SquaredExponential()
inducing_variable = gpflow.inducing_variables.InducingPoints(
    np.random.randn(num_inducing, 1),
)
gp_layer2 = gpflux.layers.GPLayer(
    kernel,
    inducing_variable,
    num_data=num_data,
    num_latent_gps=1,
    mean_function=gpflow.mean_functions.Identity(),
)

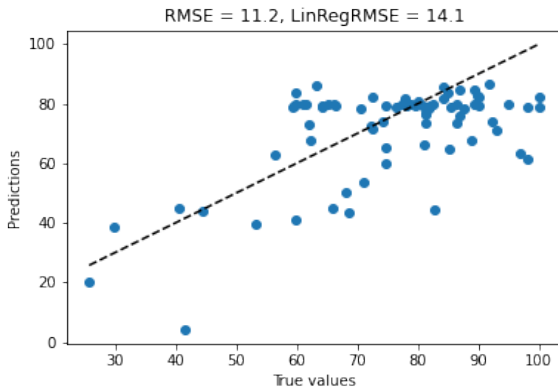
likelihood_layer = gpflux.layers.LikelihoodLayer(
    gpflow.likelihoods.Gaussian(0.01))
gpflow.set_trainable(likelihood_layer, False)
dgp = gpflux.models.DeepGP([gp_layer, gp_layer2], likelihood_layer)
```

Fitting Model

```
class PrintCallback(tf.keras.callbacks.Callback):  
    def on_epoch_end(self, epoch, logs=None):  
        if epoch % 500 == 0:  
            print(f"Epoch {epoch}: {logs}")  
  
model = dgp.as_training_model()  
model.compile(tf.optimizers.Adam(0.01))  
history = model.fit({"inputs": X, "targets": Y},  
                    epochs=int(10e3),  
                    verbose=0,  
                    batch_size=num_data, # always  
                    shuffle=False, # always  
                    callbacks=[PrintCallback()])
```

Plotting predictions

```
model = dgp.as_prediction_model()
testPreds = model(X_test)
lims = [min(Y_test),max(Y_test)]
plt.scatter(Y_test,testPreds.y_mean)
plt.plot(lims, lims, 'k--')
plt.title('RMSE = 11.2, LinRegRMSE = 14.1')
plt.xlabel('True values')
plt.ylabel('Predictions')
```



Section 2

Variational Inference

Variational Inference

There are many problems in which a posterior distribution of interest, say $p(\theta \mid \mathbf{Y})$ is difficult to obtain or costly from an MCMC perspective (where θ is all of the latent variables and parameters in our model, and $\mathbf{Y}$ is the data)

The basic idea of variational inference is to select an approximation to this posterior, say $q(\theta)$, that is tractable, and then to try to make this approximation as close as possible to the true posterior by minimizing the difference between them - usually, through the Kullback-Leibler (*KL*) divergence (relative entropy).

This reduces the inference computation to an optimization problem, which is typically more efficient, but usually an approximation.

Variational Inference (cont.)

- Let $p(\theta \mid \mathbf{Y})$ be our true (intractable) posterior distribution of interest and $q(\theta)$ some approximation from a tractable family (e.g., MVN) that has some parameters that we want to optimize to make it similar to p .
- We seek to minimize the KL divergence between these distributions.
E.g.,

$$KL(q(\theta) \parallel p(\theta \mid \mathbf{Y})) \equiv \int q(\theta) \log \frac{q(\theta)}{p(\theta \mid \mathbf{Y})} d\theta$$

Note, the KL divergence is not symmetric, so one would get a different solution if one switched the roles of q and p . The form listed here is more computationally tractable because expectations with respect to q are easily obtained (by choice). The other formulation, $KL(p \parallel q)$ is used in the expectation propagation (EP) algorithm (see Bishop, Sec. 10.7).

Variational Inference (cont.)

Let's consider the KL divergence in more detail:

$$\begin{aligned} KL(q(\theta) \| p(\theta | \mathbf{Y})) &= \int q(\theta) \log \frac{q(\theta)}{p(\theta | \mathbf{Y})} d\theta \\ &= \int q(\theta) \log \frac{q(\theta)p(\mathbf{Y})}{p(\theta, \mathbf{Y})} d\theta \\ &= \int q(\theta) \left[\log p(\mathbf{Y}) + \log \frac{q(\theta)}{p(\theta, \mathbf{Y})} \right] d\theta \\ &= \log p(\mathbf{Y}) + \int q(\theta) \log \frac{q(\theta)}{p(\theta, \mathbf{Y})} d\theta \\ &= \log p(\mathbf{Y}) - \int q(\theta) \log \frac{p(\theta, \mathbf{Y})}{q(\theta)} d\theta \end{aligned}$$

The last term on the right-hand-side is called the evidence lower bound (ELBO). Note, $KL \geq 0$ which implies $\log p(\mathbf{Y}) \geq ELBO$. Note, the KL divergence is zero when $q(\theta)$ equals the posterior of interest, $p(\theta | \mathbf{Y})$.

Variational Inference (cont.)

So, our optimization problem is:

$$\operatorname{argmax}_{q \in Q} \text{ELBO}(q)$$

Thus, we can see that this suggests that why we considered the KL in this direction - so, we can pick tractable distributions for $q \in Q$. In practice, we parameterize this, $q(\theta \mid \omega)$ and the ELBO becomes a function of these parameters, ω . One of the most common ways to choose q is by the mean field approximation, sometimes called mean-field variational Bayes (MFVB):

$$Q_{MFVB} \equiv \left\{ q : q(\theta) = \prod_{i=1}^M q_i(\theta_i) \right\},$$

where we consider the joint distribution of $q(\theta)$ to be fully factorable. In addition, it is then convenient to assume that these distributions q_i come from the exponential family (this allows conjugacy).

Variational Inference (cont.)

So, minimizing the KL divergence is equivalent to maximizing the ELBO, so under MFVB our optimization problem is:

$$\operatorname{argmax}_{q \in Q} ELBO \left(\prod_i q(\theta_i) \right).$$

How do we solve this optimization problem? First, let's re-write the lower bound in terms of this factorization (see Sec 10.1.1 in Bishop, 2006).

$$\begin{aligned} L(q) &\equiv \int q(\boldsymbol{\theta}) \log \frac{p(\mathbf{Y}, \boldsymbol{\theta})}{q(\boldsymbol{\theta})} d\boldsymbol{\theta} \\ &= \int \prod_i q_i \left[\log \frac{p(\mathbf{Y}, \boldsymbol{\theta})}{\prod_i q_i} \right] d\boldsymbol{\theta} \\ &= \int \prod_i q_i [\log p(\mathbf{Y}, \boldsymbol{\theta}) - \sum_i \log q_i] d\boldsymbol{\theta} \end{aligned}$$

Variational Inference (cont.)

$$\begin{aligned} &= \int q_j \left[\int \log p(\mathbf{Y}, \boldsymbol{\theta}) \prod_{i \neq j} q_i d\theta_i \right] d\theta_j - \int q_j \log q_j d\theta_j + \text{const.} \\ &= \int q_j \log \tilde{p}(\mathbf{Y}, \theta_j) d\theta_j - \int q_j \log q_j d\theta_j + \text{const.}, \end{aligned}$$

where

$$\log \tilde{p}(\mathbf{Y}, \theta_j) \equiv E_{i \neq j}[\log p(\mathbf{Y}, \boldsymbol{\theta})] = \int \log p(\mathbf{Y}, \boldsymbol{\theta}) \prod_{i \neq j} q_i d\theta_i,$$

where we average out all the hidden variables except θ_j . Now, say we keep $\{q_{i \neq j}\}$ fixed and maximize $L(q)$ with respect to all possible forms for $q_j(\theta_j)$. Maximizing this is equivalent to minimizing the Kullback-Leibler divergence with the minimum occurring when $q_j(\theta_j) = \tilde{p}(\mathbf{Y}, \theta_j)$. Thus, the optimal solution is given by:

$$\log q_j^*(\theta_j) = E_{i \neq j}[\log p(\mathbf{Y}, \boldsymbol{\theta})] + \text{const.} \quad (1)$$

for each $j = 1, \dots, M$.

Variational Inference (cont.)

Thus, the optimal solution for factor q_j is by taking the expectation with respect to the all of the other factors of the log of the joint distribution of all random components and the data.

Practically, we don't need to worry about the constant in (1) and just take the exponential and normalize:

$$q_j^*(\theta_j) = \frac{\exp(E_{i \neq j}[\log p(\mathbf{Y}, \theta)])}{\int \exp(E_{i \neq j}[\log p(\mathbf{Y}, \theta)]) d\theta_j}$$

In practice, people typically work with (1) since we know q_j must be a normalized distribution.

Variational Inference (cont.)

To implement, we cycle through each $j = 1, \dots, M$ using (1), always using the most updated θ_j in the expectation. Convergence is guaranteed because the bound is convex with respect to each of the factors. Numerically, this is often done with coordinate descent algorithms (note, coordinate descent is gradient descent but optimizes one parameter at a time).

Other approaches can be used more generally, especially when the mean field approximation is not used (e.g., stochastic variational inference, automatic differentiation variational inference).

Simple Example

Consider an example where we have data $\mathbf{Y} = (y_1, \dots, y_N)'$ and are interested in the population mean and precision, μ and τ , respectively (recall, precision is just the inverse of the variance; $\tau = 1/\sigma^2$); $\boldsymbol{\theta} = (\mu, \tau)'$.
Model Assumption:

$$\begin{aligned} p(\mathbf{Y} \mid \boldsymbol{\theta}) : y_i &\sim \text{iid } N(\mu, \tau^{-1}), \quad i = 1, \dots, N \\ p(\boldsymbol{\theta}) : \mu \mid \tau &\sim N(\mu_0, (\kappa_0 \tau)^{-1}), \\ \tau &\sim \text{Gamma}(a_0, b_0). \end{aligned}$$

Note, we use the mean-field approximation: $q(\mu, \tau) = q(\mu)q(\tau)$. But, note, we do not need to specify the forms for these component distributions, as they will “fall out” of the derivation (they turn out to be Gaussian and Gamma, respectively, as we will see). [Of course, we do not need variational methods to solve this problem this is just for pedagogical illustration!]

Simple Example (cont.)

We need to work out the optimum factors, $q_\mu(\mu)$ and $q_\tau(\tau)$; to do this, we need to use (1) as follows:

$$\begin{aligned}\log q_\mu^*(\mu) &= E_\tau[\log p(\mathbf{Y} \mid \mu, \tau) + \log p(\mu \mid \tau)] + \text{const.} \\ &= -\frac{1}{2} E_\tau[\tau] \left[\sum_{i=1}^N (y_i - \mu)^2 + \kappa_0 (\mu - \mu_0)^2 \right] + \text{const.}\end{aligned}$$

Completing the square with respect to μ , we can show that $q_\mu(\mu)$ is Gaussian $N(\mu_N, \kappa_N)$, with mean and variance given by:

$$\begin{aligned}\mu_N &\equiv \frac{\kappa_0 \mu_0 + N \bar{y}}{\kappa_0 + N} \\ \kappa_N &= (\kappa_0 + N) E_\tau[\tau]\end{aligned}$$

Simple Example (cont.)

The optimal solution for $q_\tau(\tau)$ is given by:

$$\begin{aligned}\log q_\tau^*(\tau) &= E_\mu [\log p(\mathbf{Y} \mid \mu, \tau) + \log p(\mu \mid \tau)] + \log p(\tau) + \text{const.} \\ &= (a_0 - 1) \log \tau - b_0 \tau + (1/2) \log \tau + (N/2) \log \tau \\ &\quad - (\tau/2) E_\mu \left[\sum_{i=1}^N (y_i - \mu)^2 + \kappa_0 (\mu - \mu_0)^2 \right] + \text{const.}\end{aligned}$$

This is just the log of Gamma distribution, say $\text{Gam}(a_N, b_N)$, where

$$\begin{aligned}a_N &= a_0 + \frac{N + 1}{2} \\ b_N &= b_0 + \frac{1}{2} E_\mu \left[\sum_{i=1}^n (x_n - \mu)^2 + \kappa_0 (\mu - \mu_0)^2 \right].\end{aligned}$$

Simple Example (cont.)

Note, the optimal distributions for q_μ and q_τ were not specified, but were a consequence of the mean field assumption and the conjugate priors. Note, that each of these depends on the moments evaluated with respect to the other distribution. We can proceed as follows:

- Make an initial guess of $E_\tau[\tau]$ and use this to re-compute the distribution $q_\mu[\mu]$
- Given this revised distribution, extract the required moments $E[\mu]$ and $E[\mu^2]$
- Use these to recompute $q_\tau(\tau)$, and so on

To implement these updates, we have to compute the various expectations. First, note we have:

$$\begin{aligned}E_\mu[\mu] &= \mu_N \\E_\mu[\mu^2] &= \frac{1}{\kappa_n} + \mu_n^2 \\E_\tau[\tau] &= \frac{a_N}{b_N}\end{aligned}$$

Simple Example (cont.)

Now, we can specify explicit forms for the update equations: For $q(\mu)$:

$$\mu_N = \frac{\kappa_0 \mu_0 + N \bar{y}}{\kappa_0 + N}$$
$$\kappa_N = (\kappa_0 + N) \frac{a_N}{b_N}$$

For $q(\tau)$:

$$a_N = a_0 + \frac{N + 1}{2}$$
$$b_N = b_0 + \kappa_0 \left(E[\mu^2] + \mu_0^2 - E[\mu]\mu_0 \right) + (1/2) \sum_{i=1}^N \left(y_i^2 + E[\mu^2] - 2E[\mu]y_i \right)$$

Note, μ_N and a_N are constants and don't need to be updated!

Simple Example (cont.)

Bishop (2006, Fig 10.4) shows an implementation of this simple example.

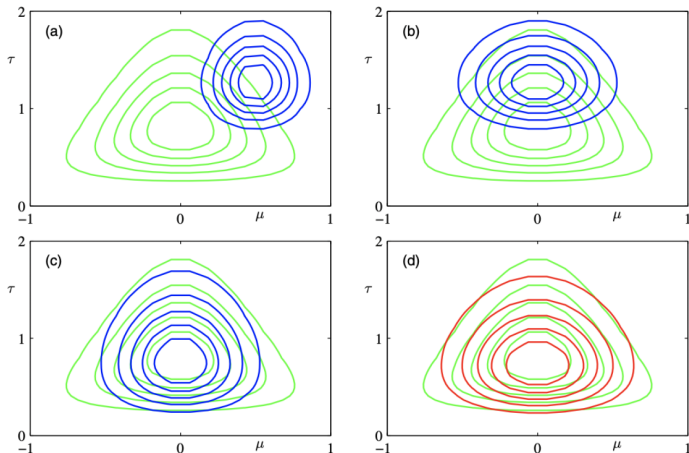


Figure 10.4 Illustration of variational inference for the mean μ and precision τ of a univariate Gaussian distribution. Contours of the true posterior distribution $p(\mu, \tau | D)$ are shown in green. (a) Contours of the initial factorized approximation $q_\mu(\mu)q_\tau(\tau)$ are shown in blue. (b) After re-estimating the factor $q_\mu(\mu)$. (c) After re-estimating the factor $q_\tau(\tau)$. (d) Contours of the optimal factorized approximation, to which the iterative scheme converges, are shown in red.

Section 3

Bayesian Neural Networks

Motivation

Typical NN models are prone to overfitting. Oftentimes when we receive a predicted output we have the illusion that the model is perfectly confident in this result.

This is because there is no built in uncertainty quantification with neural network models.

- weights are point estimates only.

Blundell et. al. 2015 introduced the concept of weight uncertainty in neural networks and demonstrated that quantifying stochasticity has a similar regularization effect as dropout does in a standard NN.

Moving towards stochasticity

We can obtain the MLE of the weights $\mathbf{w}^{\text{MLE}}$ of a NN given a set of training examples $\mathcal{D} = (\mathbf{x}_i, \mathbf{y}_i)_i$ as

$$\begin{aligned}\mathbf{w}^{\text{MLE}} &= \arg \max_{\mathbf{w}} \log P(\mathcal{D} \mid \mathbf{w}) \\ &= \arg \max_{\mathbf{w}} \sum_i \log P(\mathbf{y}_i \mid \mathbf{x}_i, \mathbf{w}).\end{aligned}$$

Typically, we find these weights using SGD. Regularization can be introduced by placing a prior upon the weights $\mathbf{w}$ and finding the maximum a posteriori (MAP) weights $\mathbf{w}^{\text{MAP}}$:

$$\begin{aligned}\mathbf{w}^{\text{MAP}} &= \arg \max_{\mathbf{w}} \log P(\mathbf{w} \mid \mathcal{D}) \\ &= \arg \max_{\mathbf{w}} \log P(\mathcal{D} \mid \mathbf{w}) + \log P(\mathbf{w}).\end{aligned}$$

- Gaussian prior yields $L2$ regularization
- Laplace prior yields $L1$ regularization

Naturally, optimizing this directly is computationally impractical.

Variational inference to the rescue

We approximate the true posterior $P(\mathbf{w} \mid \mathcal{D})$ with $q(\mathbf{w} \mid \theta)$

$$\begin{aligned}\theta^* &= \arg \min_{\theta} \text{KL}[q(\mathbf{w} \mid \theta) \parallel P(\mathbf{w} \mid \mathcal{D})] \\ &= \arg \min_{\theta} \int q(\mathbf{w} \mid \theta) \log \frac{q(\mathbf{w} \mid \theta)}{P(\mathbf{w})P(\mathcal{D} \mid \mathbf{w})} d\mathbf{w} \\ &= \arg \min_{\theta} \text{KL}[q(\mathbf{w} \mid \theta) \parallel P(\mathbf{w})] - \mathbb{E}_{q(\mathbf{w} \mid \theta)}[\log P(\mathcal{D} \mid \mathbf{w})]\end{aligned}$$

The cost in the last line is approximated by

$$\mathcal{F}(\mathcal{D}, \theta) \approx \sum_{i=1}^n \log q(\mathbf{w}^{(i)} \mid \theta) - \log P(\mathbf{w}^{(i)}) - \log P(\mathcal{D} \mid \mathbf{w}^{(i)})$$

where each $\mathbf{w}^{(i)}$ denotes a Monte Carlo sample drawn from the variational posterior $q(\mathbf{w}^{(i)} \mid \theta)$.

Learning means and variances

Idea: We will learn ideal values for *means* and *standard deviations* of normal distributions in place of standard weights.

If our posterior for the weights $q(\mathbf{w} \mid \theta)$ are independent normals, we can sample from standard normals and shift by a mean μ and scaling by a standard deviation σ .

- NNs like to learn values that exist on the real line, so the standard deviation is parameterized by $\sigma = \log(1 + \exp(\rho))$.

The model will then learn $\theta = (\mu, \rho)$.

During the forward passes of this model, standard normals are samples and shifted according to the learned parameters.

As we will see, for every “neuron” in the network the model will learn *two* values instead of one, which leads to a doubling of the number of weights. This is often a fair tradeoff for what we get (stochasticity).

Algorithm

- 1 Sample $\epsilon \sim \mathcal{N}(0, I)$
- 2 Let $\mathbf{w} = \mu + \log(1 + \exp(\rho)) \circ \epsilon$.
- 3 Let $\theta = (\mu, \rho)$.
- 4 Let $f(\mathbf{w}, \theta) = \log q(\mathbf{w} | \theta) - \log P(\mathbf{w})P(\mathcal{D} | \mathbf{w})$.
- 5 Calculate the gradient with respect to the mean

$$\Delta_{\mu} = \frac{\partial f(\mathbf{w}, \theta)}{\partial \mathbf{w}} + \frac{\partial f(\mathbf{w}, \theta)}{\partial \mu}$$

- 6 Calculate the gradient with respect to the standard deviation parameter ρ

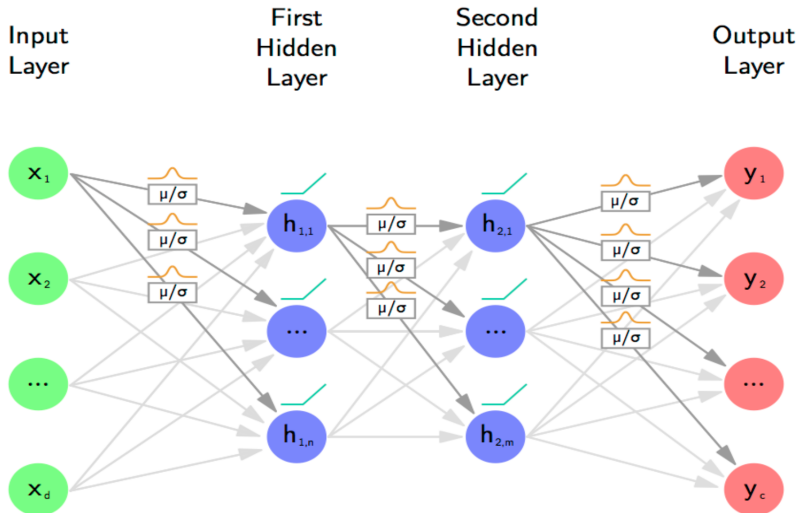
$$\Delta_{\rho} = \frac{\partial f(\mathbf{w}, \theta)}{\partial \mathbf{w}} \frac{\epsilon}{1 + \exp(-\rho)} + \frac{\partial f(\mathbf{w}, \theta)}{\partial \rho}.$$

- 7 Update the variational parameters:

$$\mu \leftarrow \mu - \alpha \Delta_{\mu}$$

$$\rho \leftarrow \rho - \alpha \Delta_{\rho}.$$

BNN Diagram



Usage in Tensorflow, an example

Using this methodology in Tensorflow is very simple. We just need to access the DenseReparameterization layer of the tensorflow-probability library.

```
import numpy as np
import tensorflow as tf
import tensorflow_probability as tfp
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import fetch_california_housing
import matplotlib.pyplot as plt

tfd = tfp.distributions
tfpl = tfp.layers
```

Example: Loading data

Load the dataset

```
data = fetch_california_housing()
```

```
X, y = data.data, data.target
```

Split the dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y,  
                                                    test_size=0.3)
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
y_train = y_train.reshape(-1, 1) # column vector
```

```
y_test = y_test.reshape(-1, 1)
```

Example: Define Model

```
def bnn(input_shape, KLden):
    inputs = tf.keras.layers.Input(shape=input_shape)
    x = tfpl.DenseReparameterization(
        32,
        activation='relu',
        kernel_prior_fn=tfpl.default_multivariate_normal_fn,
        kernel_posterior_fn=tfpl.default_mean_field_normal_fn(),
        kernel_divergence_fn=lambda q, p, _: tfd.kl_divergence(q, p) / KLden
    )(inputs)
    outputs = tfpl.DenseReparameterization(
        1,
        kernel_prior_fn=tfpl.default_multivariate_normal_fn,
        kernel_posterior_fn=tfpl.default_mean_field_normal_fn(),
        kernel_divergence_fn=lambda q, p, _: tfd.kl_divergence(q, p) / KLden
    )(x)
    model = tf.keras.Model(inputs=inputs, outputs=outputs)
    return model

bayesian_NN = bnn(X_train.shape[1:], X_train.shape[0])
```

- The layerDenseFlipout is similar, with some computational improvements
- Convolution2D<Flipout/Reparameterization> adds this functionality to convolutional layers in CNNs.

Model summary

```
bayesian_NN.summary()
```

```
Model: "model"
```

Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	[(None, 8)]	0
dense_reparameterization (DenseReparameterization)	(None, 32)	544
dense_reparameterization_1 (DenseReparameterization)	(None, 1)	65
=====		
Total params: 609		
Trainable params: 609		
Non-trainable params: 0		

Example: Build model

```
bayesian_NN.compile(  
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),  
    loss=tf.keras.losses.MeanSquaredError()  
)  
  
history = bayesian_NN.fit(  
    X_train, y_train,  
    epochs=20,  
    batch_size=32,  
    validation_data=(X_test, y_test),  
    verbose=1  
)
```

Predictions

```
✓ bayesian_model.predict(X_train[0,:].reshape(1,-1), verbose=0) ...  
array([[1.9971247]], dtype=float32)
```

```
✓ bayesian_model.predict(X_train[0,:].reshape(1,-1), verbose=0) ...  
array([[1.9453385]], dtype=float32)
```

Histogram of many predictions

This is the result of passing the same input in for prediction repeatedly.

